

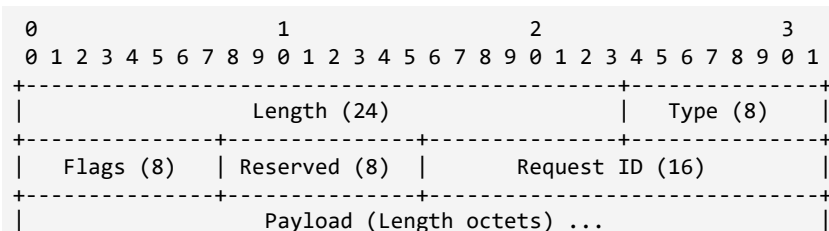
BHTTP/1 – HTTP semantics in binary frames

Status: version 1. The key words MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. All integers are unsigned and big-endian (network byte order).

1. Connection model

A client opens **one TCP connection** to the server and reuses it for every request. There is no handshake or preface: the first octet the client sends is the start of a frame. For every request the server sends exactly one response, **in the order the requests were received**. A client MAY send the next request before the previous response has finished; it matches responses by order and checks the Request ID. Either side MAY close the connection between messages. A server MAY close a connection that has been idle for 60 seconds. Closing in the middle of a message means that message failed.

2. Frame header (8 octets, fixed)



Field	Bits	Meaning
Length	24	Payload length in octets, header excluded. MUST NOT exceed 16384 in v1.
Type	8	Frame type (§3).
Flags	8	Bit 0x01 = END : last frame of this message. Other bits MUST be sent as 0 and MUST be ignored.
Reserved	8	MUST be sent as 0x00 and MUST be ignored on receipt.
Request ID	16	Ties frames to a request. 1–65535 for messages; 0 = the connection itself (§6).

Why these widths. HTTP/2 uses 24/8/8/31 (+1 reserved bit). We keep its first three and change the last. *Length 24:* v1 caps frames at 16 KiB so a receiver never buffers much and can check the cap before reading a payload. The field is still 24 bits wide so a later version can raise the cap without changing the header, which is HTTP/2's SETTINGS_MAX_FRAME_SIZE argument. A 16-bit field would cap frames at 64 KiB forever. A 32-bit one costs an octet and lets a corrupt header announce a 4 GiB frame. The cap also catches peers that are not speaking BHTTP: the text GET decodes as a 4.6 MB frame and is rejected at once instead of hanging. *Type 8, Flags 8:* we use two types and one flag, and the rest is room to grow. *Request ID 16:* HTTP/2 needs 31 bits because streams run concurrently and an ID is never reused on a connection. BHTTP answers strictly in order, so the ID only correlates and detects desynchronisation, and it may wrap from 65535 to 1. *Reserved 8:* it pads the header to 8 octets, exactly half a hexdump line. It also gives a v2 eight bits (for example a priority) that v1 receivers already ignore.

3. Frame types

Type	Name	Payload
0x00	—	Reserved, never assigned, so a zero-filled buffer is not a valid frame.
0x01	HEADERS	A header block (§4). Starts every request and every response.
0x02	DATA	Message body octets, opaque.
0x03–0xFF	—	Unassigned.

A receiver meeting a frame type it does not know MUST skip it cleanly. It reads and discards exactly Length payload octets, whatever the Flags or Request ID. The frame does not end a message, change any state or produce an error, and the receiver continues with the next frame. This rule is how BHTTP grows: a v2 can add frame types, such as settings, a dynamic header table or a ping, that v1 peers silently ignore. That is why v1 has no version number.

4. Header block (HEADERS payload)

A header block is a sequence of fields that runs to the end of the payload. There is no count and no terminator. Each field is:

```
+-----+ if Index = 0: +-----+-----+-----+-----+
| Index (8) |          | NameLen (8) | Name (NameLen) | | ValueLen (16) | Value ... |
+-----+          +-----+-----+-----+-----+
```

Index 1–10 names a field from the static table below, so the name costs one octet. **Index 0** means a literal name follows, length-prefixed by one octet, 1–255 octets of lowercase ASCII. **Index 11–255** is reserved, and a receiver **MUST** treat it as malformed. Values are 0–65535 opaque octets, normally ASCII. A sender **SHOULD** use the index when the name is in the table, and a receiver **MUST** accept either form. Receivers compare names without regard to case. A field **MUST NOT** run past the end of the payload.

1 :method	2 :path	3 :status	4 host	5 user-agent
6 accept	7 content-type	8 content-length	9 server	10 date

These are the ten names that bcurl and bserve actually send. That is HPACK's static table and length-prefixed literals, its first two mechanisms, without Huffman coding or a dynamic table.

5. Messages

Request: one HEADERS frame, then zero or more DATA frames (the body). The last frame of the request carries END, so a body-less request is a single HEADERS frame with END. The block **MUST** contain :method and :path exactly once each and **MUST NOT** contain :status or any other :-name. :path **MUST** start with /. It may carry %xx escapes and a ?query, which a file server ignores. The client chooses the Request ID, normally 1 upwards, wrapping 65535 to 1. v1 servers support GET and HEAD.

Response: one HEADERS frame carrying :status as exactly three ASCII digits (200), then zero or more DATA frames, with END on the last frame. Every frame of a response carries the Request ID of its request. A response to HEAD has no DATA frames. content-length is advisory. **END alone delimits the body.** Senders **SHOULD** send DATA in frames of at most 16384 octets.

Why ASCII :status: every value has one encoding, so a decoder needs no per-name rules. The cost is one octet more than a binary u16.

6. Errors

Condition	Server reply	Connection
Malformed header block: truncated field, Index 11–255, or NameLen 0	400 on that Request ID	stays open
Missing or duplicate :method/:path, a :status or unknown :-name, or :path not starting with /	400 on that Request ID	stays open
DATA frame for an ID with no open request, or a new HEADERS frame before the open request's END	400 on that ID	stays open
Path not found, or resolving outside the root (.., links)	404	stays open
Method other than GET/HEAD	405 with an allow field	stays open
Length > 16384, HEADERS on Request ID 0, or EOF inside a frame	400 on Request ID 0 , best effort	closed

A message-level error leaves the framing intact, because Length says exactly where the next frame starts. The server therefore answers and carries on. A connection-level error means frame boundaries can no longer be trusted, so the server reports it on ID 0 and closes. A client receiving a response on an unexpected Request ID, a second HEADERS frame in one response, DATA before HEADERS, or Length > 16384 **MUST** treat it as a connection error and close.

7. Example: the smallest request, GET / as Request ID 1 (18 octets)

```
00 00 0a | 01 | 01 | 00 | 00 01 || 01 00 03 47 45 54 || 02 00 01 2f
Length=10 HDRS END  rsv  ID=1  || :method len3 "GET" || :path len1 "/"
```

A complete annotated request and response is in `HEXDUMP.md`.